# Prompt 工程系统学习报告

## 理论方法 评测体系与 KARMA 知识抽取实践

阶段学习报告

汇报人 _____

日期 2026 年 9 月

# 摘要

本报告围绕 Prompt 工程的概念边界、设计方法、评测体系、安全风险与知识抽取应用展开系统梳理。学习材料从入门教程扩展到官方技术文档、综述论文、代表性研究、评测框架和安全指南，并结合 KARMA 项目现有实体发现与关系抽取提示词进行案例分析。报告的核心判断是：高质量 Prompt 不是一段修辞精巧的指令，而是一份可执行、可验证、可迭代的任务规格；Prompt 工程的成熟形态也不只是措辞优化，而是由任务定义、上下文组织、结构化接口、评测数据和运行治理共同构成的系统工程。

在方法层面，本报告区分零样本与少样本提示、思维链与任务分解、自洽采样、程序辅助推理、检索增强、工具调用和结构化输出等技术，并强调这些方法并非普遍有效，必须依据任务类型、模型特性、成本限制和失败模式进行选择。在应用层面，报告将上述原则落到知识图谱构建场景，指出 KARMA 当前 Prompt 的主要改进空间集中在规则冲突、证据边界、复合实体原子化、模态与否定表达、两阶段修订锚定以及模型自评偏差等方面。最后，报告提出一套可直接执行的对照实验与组会 PPT 叙事框架，使后续学习从“阅读更多材料”转向“用证据验证设计假设”。

**关键词** Prompt 工程 上下文工程 大语言模型评测 结构化输出 知识抽取 KARMA

# 阅读导航

第一章 学习目标与研究范围
第二章 Prompt 工程的概念与发展
第三章 Prompt 设计的核心方法
第四章 上下文工程与知识接入
第五章 评测驱动的 Prompt 开发
第六章 安全 鲁棒性与运行治理
第七章 信息抽取与知识图谱场景
第八章 KARMA 项目 Prompt 案例分析
第九章 核心结论与认知升级
第十章 后续学习与组会汇报设计
附录 术语速查与学习资源
参考文献

# 第一章 学习目标与研究范围

## 学习背景

此前接触的 Prompt 工程内容多以技巧清单为主，例如指定角色、补充上下文、要求分步思考或提供示例。这些做法有一定实用价值，但如果缺少任务定义、评价标准与失败分析，就容易出现“同一批技巧换一种说法”的重复感。本阶段学习因此不再以收集模板为目标，而是试图回答更基础的问题：模型为何会受提示影响，提示怎样与数据、工具和输出接口共同工作，效果如何被可靠测量，以及一个 Prompt 如何从个人经验转化为团队可复用资产。

## 学习范围与资料方法

本报告所依据的学习材料分为五类：官方文档用于把握当前产品能力与推荐实践；综述用于建立统一术语和技术地图；代表性论文用于理解方法的原始问题、实验设置与适用边界；评测与安全资料用于补足教程中常被忽略的可靠性问题；KARMA 本地 Prompt 与项目文档用于验证理论能否解释真实工程现象。Prompt Report 对 33 个相关术语和 58 类文本提示技术进行了系统梳理，说明 Prompt 工程已经形成较大的方法谱系，但方法数量本身并不等于工程成熟度。[1]

**表一 学习资料的功能分工**

| 资料类型 | 主要用途 | 本报告的使用方式 |
| --- | --- | --- |
| 官方文档 | 确认当前能力 边界与接口 | 核对推理模型提示 结构化输出和评测流程 |
| 综述与教程 | 建立知识地图和共同术语 | 整理方法家族 选择学习主线 |
| 代表性论文 | 理解方法提出的真实问题 | 关注实验条件 收益来源与失效边界 |
| 评测与安全资料 | 识别平均分之外的风险 | 设计失败分类 注入防护与回归测试 |
| 项目材料 | 把理论转化为可验证问题 | 分析 KARMA 实体和关系 Prompt 并提出实验 |

## 报告重点

本报告不追求平均介绍所有技巧，而把重点放在三个层次。第一层是 Prompt 作为任务规格的基本结构，决定模型是否理解目标、约束与输出契约；第二层是评测驱动的迭代流程，决定改动是否真正提升质量；第三层是知识抽取场景中的证据性、原子性和一致性，决定结果能否进入后续知识图谱。

本阶段成果属于文献梳理与项目设计分析。文中展示的提示词示例用于解释方法，实验矩阵用于安排后续验证；本报告没有把这些建议标记为已经完成的模型对照实验。阅读时可以先看第二章建立概念，再看第五章理解评价标准，最后结合第八章形成项目问题和实验假设。

# 第二章 Prompt 工程的概念与发展

## Prompt 工程的定义

Prompt 工程可以定义为围绕大语言模型输入进行设计、测试、评估和维护的过程，其目标是在给定模型、数据与成本约束下稳定实现任务行为。这里的 Prompt 不仅是用户提出的问题，还包括系统指令、上下文材料、示例、工具说明、输出模式和对话历史。因此，一个生产级 Prompt 更接近“自然语言编写的接口契约”，而不是一句灵感式命令。

## 模型为什么会受 Prompt 影响

大语言模型根据当前上下文预测后续内容，Prompt 改变了模型所处的条件分布。指令告诉模型要完成什么任务，示例提供任务在当前上下文中的局部模式，检索材料提供可引用事实，输出模式限制结果的外部形态。由于同一语义可以有多种表达，而模型训练目标并不等同于用户的业务指标，提示措辞、示例顺序、上下文位置甚至无关文本都可能改变输出。研究发现少样本示例的排列顺序会显著影响结果，长上下文中的关键信息放在中部时也可能更难被利用。[6][11]

## 从提示词技巧到上下文与评测工程

Prompt 工程的发展重心正在从“如何写一句更聪明的话”转向“如何组织模型一次决策所需的全部信息”。当系统包含检索、记忆、工具、多个智能体和结构化接口时，真正需要设计的是上下文的选择、排序、压缩、可信度和生命周期，这通常被概括为上下文工程。同时，模型或版本变化会导致既有 Prompt 性能漂移，单靠主观阅读无法判断好坏，因此评测工程成为另一条主线。成熟系统的基本循环是先定义成功标准，再建立测试集，随后修改 Prompt 并进行回归验证，而不是凭少数成功样例确定方案。

## 技术边界

Prompt 并不能弥补所有问题。知识缺失通常需要检索或外部数据库，确定性计算适合交给代码和工具，稳定格式需要结构化输出约束，复杂业务规则可能需要拆分工作流，而高风险结论还需要人工

复核。若一个问题本质上来自脏数据、标签定义不清、检索召回不足或模型能力不匹配，继续增加提示文本往往只会提高复杂度。识别何时不该继续调 Prompt，本身就是 Prompt 工程的重要能力。

### 提示设计与参数训练的区别

本报告讨论的提示设计主要是在不更新模型参数的前提下改变输入。少样本提示属于上下文中的学习，示例只影响当前调用，不会自动变成模型的永久知识。与之不同，监督微调使用训练样本更新参数；Prompt Tuning 通常指学习连续的提示向量，也不等同于人工修改自然语言指令。阅读论文时必须分清效果来自输入设计、专门训练、模型升级还是更多计算资源，否则容易把无法直接复现的训练收益误认为可复制的提示技巧。[1]

## 第三章 Prompt 设计的核心方法

### 把任务写成可执行规格

一个稳健 Prompt 首先需要把含糊目标转换为可执行规格。规格至少应说明任务目标、输入边界、允许与禁止的行为、判断标准、输出字段以及异常情况的处理方式。例如“抽取重要实体”并不可测，因为“重要”和“实体”的边界没有定义；改写为“仅抽取原文中可定位的生物医学实体，保持原文跨度，排除纯数量与无独立语义修饰词，并按给定类型返回”后，任务才具备一致标注和评测的可能。

### 指令 上下文 示例与输出契约

Prompt 可以拆为四个相互依赖的组成部分。指令规定行动和优先级；上下文提供完成任务所需的事实；示例把抽象规则转化为局部行为参照；输出契约规定字段、类型和失败返回。四者的顺序不是唯一的，但职责应清晰，外部材料与指令应有明显边界。对长任务而言，先给出简短目标，再给规则与数据，最后重复最关键的输出要求，通常比在单段文字中混合所有信息更便于维护。

### 零样本与少样本提示

零样本提示适合规则清晰、模型已有较强先验的任务，维护成本低，但容易暴露标签边界不一致的问题。少样本提示通过示例校准输出风格和决策边界，尤其适合分类、抽取和规范化任务；其关键不在示例数量，而在覆盖典型样例、边界样例、反例和易混淆样例。示例还应避免泄露测试答案或携带与任务无关的表面规律。由于示例顺序会引起波动，最终方案需要在多种顺序和不同样本上验证，而不能只选择一次效果最好的排列。[6]

## 推理 分解与自洽

思维链提示通过展示中间步骤，在论文测试的算术、常识和符号推理任务中改善了大模型表现。[2] 自洽方法采样多条推理路径，按最终答案聚合结果，适用于答案可归并的任务；多次回答可能共享偏差，因此多数一致并不保证正确。[3] Least-to-Most 先分解问题，再顺序利用子问题答案。[4] OpenAI 推理模型指南建议优先提供直接指令、明确约束与成功标准，不必机械要求“逐步思考”。[18] 这不能推广为所有模型的统一规则；模型生成的解释也不能直接作为其真实内部推理的证据。

## 程序辅助推理与工具使用

当任务包含精确计算、数据库查询或外部操作时，将可验证步骤交给工具通常比要求模型纯文本推理更可靠。PAL 让模型生成程序并由执行器完成计算；ReAct 把推理与行动交替组织，使模型可以查询环境、观察结果并继续决策。[5][7] 工程上的关键是明确工具何时可用、参数如何构造、调用失败如何处理以及最终答案如何引用工具结果。工具调用不是让模型“更聪明”，而是把模型擅长的语义判断与外部系统擅长的确定性操作分开。

## 结构化输出

结构化输出把自然语言响应限制为预定义模式，可显著减少 JSON 语法错误、字段缺失和类型漂移，并便于下游程序消费。当前官方文档建议在需要稳定机器接口时使用 JSON Schema 等约束，而不是只在 Prompt 中写“请返回 JSON”。[20] 但结构正确不代表语义正确：实体可以被放进合法字段却仍然抽错，关系可以符合枚举却没有文本证据。因此结构化输出解决的是接口层可靠性，事实性和任务正确性仍需独立评测。

响应还可能因为拒绝处理、达到输出限制或调用失败而不完整。应用程序应先检查调用状态，再验证模式和业务规则；对知识抽取，还要核对证据是否确实出现在源文本中、关系端点能否定位，以及相同事实是否重复。分隔符、JSON Schema 和输出校验分别承担不同职责，不能互相替代。

## 自动 Prompt 优化的基本思想

Automatic Prompt Engineer 将指令设计转为候选生成、评分与搜索过程：模型提出多种指令，在一组任务样例上测量效果，再选出更好的候选。[10] 这一方向的意义在于把人工试错部分自动化，但它仍会过拟合开发样例。实际使用时应保留独立测试集，给搜索设置调用预算，同时保存候选及评分记录；如果只奖励格式通过率，优化器就可能学会输出格式完美但内容空洞的答案。

表二 主要方法的用途与边界

| 方法 | 主要解决的问题 | 常见风险 | 适合的验证方式 |
| --- | --- | --- | --- |
| 少样本提示 | 标签边界和输出风格不清 | 顺序敏感 示例偏置 | 换序测试 边界集测试 |
| 思维链与任务分解 | 多步推理和组合困难 | 冗长 错误链条被放大 | 按难度分层测试 |
| 自洽采样 | 单次推理偶然性 | 成本升高 多数答案也可能错 | 准确率 成本 延迟联合评估 |
| 工具调用 | 计算 查询和外部动作 | 参数错误 工具结果误读 | 执行成功率 结果一致性 |
| 结构化输出 | 字段和格式不稳定 | 合法格式掩盖语义错误 | 模式通过率加任务指标 |

# 第四章 上下文工程与知识接入

## 上下文工程的对象

上下文工程关注模型在一次决策中究竟看见什么，以及这些信息以什么顺序、可信度和粒度出现。系统指令、历史对话、检索片段、用户输入、工具返回、记忆和中间结果都会竞争有限的注意力。工程目标不是把所有材料都塞进窗口，而是让当前决策所需的高价值信息更容易被发现，同时减少过期、重复、互相冲突和不可信内容。

## 长上下文并不等于可靠记忆

更长的上下文窗口扩大了可输入的信息量，但没有自动解决信息利用问题。“Lost in the Middle”研究表明，模型对长上下文不同位置的信息利用并不均匀，关键证据位于开头或结尾时往往优于位于中部。[11] 因此长文任务仍需检索、分块、排序和摘要。对知识抽取而言，如果定义、示例、源文本和历史结果同时存在，应优先保证当前源文本与核心规则靠近决策位置，并明确历史结果只能作为待审对象，而不是事实证据。

## 检索增强生成

检索增强生成通过在推理前检索外部知识，为模型提供可追溯依据。它适合知识更新快、材料规模大或答案必须依赖私有文档的任务。检索质量需要分别考察召回、排序、片段完整性和答案引用，不能把最终回答错误全部归因于 Prompt。如果正确证据从未进入上下文，生成模型再强也无法稳定恢复；反之，检索到大量相似但冲突的片段，也可能使模型产生貌似有依据的错误综合。

### 上下文的生命周期

多轮或多智能体系统还需要管理上下文的生命周期。短期状态应记录当前任务和已完成步骤，长期记忆只保留经验证且有复用价值的信息，工具结果要附带来源和时间，模型生成的中间结果则应标记为推断而非证据。KARMA 的多智能体流程尤其需要区分“原始文献内容”“上游智能体的候选输出”和“经过验证的知识”，否则错误会在代理之间被继承，并因重复出现而获得虚假的可信度。

## 第五章 评测驱动的 Prompt 开发

### 评测为什么是核心

Prompt 优化最容易陷入的误区是凭少量演示样例和阅读感受判断效果。语言输出具有随机性，测试样例也可能恰好迎合某种措辞；没有固定数据集和指标，任何改动都无法区分真实提升与偶然波动。官方评测指南建议从目标、数据集、指标和持续评测流程出发，把评测贯穿开发和上线阶段。[19] 因而 Prompt 工程的顺序应当是先定义什么算成功，再写初版 Prompt，随后根据失败类型进行有针对性的修改。

### 评测集与数据切分

有效评测集需要同时覆盖常规样例、边界样例、对抗样例和真实失败样例，并将开发集与最终测试集分离。知识抽取任务还应按实体类型、关系类型、句子复杂度、否定和模态、并列结构、跨句依赖等维度分层统计。这样才能看出总体 F1 提升是否以牺牲某类关键事实为代价。线上出现的新错误应进入回归集，使系统不再重复犯同一种错误。

### 指标与错误分类

单一平均分不足以描述 Prompt 质量。抽取任务应至少报告精确率、召回率、F1、跨度完全匹配率、结构合法率和关键类型的分组结果；生产系统还应记录延迟、Token 成本、拒答率和稳定性。更重要的是建立错误分类，例如漏抽、边界过宽、过度拆分、关系方向错误、否定误判、模态事实化、无证据补全和格式错误。错误分类把“结果不好”转换为可修正原因，也是连接 Prompt 修改与模型行为的桥梁。

精确率表示输出项中正确项的比例，召回率表示标准答案中被找回的比例，F1 是二者的调和平均。假设标准答案有 10 条事实，模型输出 8 条，其中 6 条完全正确，则精确率为 75%，召回率为 60%，F1 约为 66.7%。这里的关键不是公式，而是先规定“正确”的匹配规则：实体按原文跨度匹配，关系按头实体、关系和尾实体共同匹配；如允许语义等价，还需明确模态、否定和数量范围是否必须一致。

数据切分应尽量按文档进行，避免同一文档的相似段落同时进入开发集与测试集。用于调整 Prompt 的失败样例属于开发数据，不能继续作为完全独立的最终验证。报告应同时给出样本数量、逐类指标和不确定性；重复调用反映模型随机性，按文档重采样则反映测试集抽样误差，两者回答的问题不同。小样本探索只能支持初步判断，不能用微小分差宣布普遍优势。

### 模型评审的价值与限制

使用大模型作为评审可以降低开放式任务的评测成本，适合比较解释质量、风格或语义一致性，但它不是绝对裁判。研究显示 LLM 评审可能存在位置偏差、冗长度偏好、自我偏好和对提示措辞敏感等问题。[13][14] 更稳妥的做法是优先使用可确定计算的指标；必须使用模型评审时，提供清晰评分量表、交换候选顺序、使用多次评审或多评审者，并用人工标注子集校准。特别要避免让生成答案的同一模型直接给自己的答案打高分而没有外部核验。

### 迭代与版本管理

Prompt 应像代码一样具有版本、变更说明和回归记录。每次修改应对应一个明确假设，并在固定评测集上比较基线与新版本，同时保存逐样例差异，检查修复与退化。只有质量、成本和延迟达到门槛，改动才进入主流程。

## 第六章 安全 鲁棒性与运行治理

### 提示注入

提示注入发生在不可信输入试图改变系统原有指令时。攻击内容可能直接来自用户，也可能隐藏在网页、文档、邮件或检索片段中。OWASP 将其列为生成式 AI 应用的重要风险，并强调仅靠一句“忽略恶意指令”不能形成可靠防护。[17] 对知识抽取系统而言，论文原文、网页内容和数据库字段都必须被视为数据；Prompt 应明确数据边界，工具权限应遵循最小化原则，高影响动作应要求独立校验或人工确认。

### 鲁棒性与分布变化

Prompt 可能在原始测试集上表现良好，却在换一种写法、语言、文体或模型版本后退化。近期研究进一步指出，提示优化结果具有明显的模型与任务依赖性，细微措辞变化也可能改变排名。[12] 因此鲁棒性测试应包括同义改写、示例换序、格式扰动、无关文本插入、长短文变化和跨模型复测。对高风险场景，报告最好给出分布外样例的失败上限，而不是只展示最佳结果。

### 运行治理

生产环境中的 Prompt 治理包括访问控制、日志、版本追踪、隐私处理、失败降级和人工复核。日志应记录模型版本、Prompt 版本、输入来源、工具调用和结构校验结果，同时避免保存不必要的敏感数据。系统还需要明确何时拒答、何时重试、何时切换备用模型，以及何时把结果交给人工。治理的价值在于把偶发的模型输出纳入可追责流程，而不是假设一次成功调用就等于可靠系统。

## 第七章 信息抽取与知识图谱场景

### 任务特征

信息抽取与一般问答的不同在于，输出必须同时满足证据约束、边界约束、类型约束和结构约束。一个三元组即使常识上正确，只要原文没有表达，也不应被视为正确抽取；一个实体名称大体相同，但跨度包含多余修饰语，也会影响后续对齐和去重。知识图谱构建还会放大局部错误：错误实体可能生成多条错误关系，并进一步污染模式对齐和冲突消解。

### 开放抽取与模式约束

传统开放信息抽取强调从文本中发现关系而不预设固定模式，适合探索未知结构，但输出异构、难以直接对齐。[15] 固定模式抽取便于评价和入库，却可能遗漏新实体类型或新关系。EDC 提出 Extract Define Canonicalize 流程，先开放抽取，再生成定义并进行规范化，试图兼顾发现能力与模式一致性。[9] 这说明 Prompt 设计不能只选择“开放”或“封闭”，更可行的是把发现、定义、规范化和验证拆成不同阶段。

### 用指南而不只是标签

GoLLIE 通过标注指南遵循方面的微调，使模型能够利用输入指南处理未见过的抽取模式。[8] 因此其零样本结果指向目标模式的泛化，不表示底座模型未经相关训练，也不能把全部收益归因于增加一段提示词。可迁移的设计启示是：仅提供 Gene、Disease 或 Regulates 等标签名称通常不足以界定任务，指南还应包含语义定义、判定边界与例外。将这一思想用于 KARMA 仍需要单独验证，尤其要确认指南适合当前文档领域。

### KARMA 方法的启示

KARMA 论文描述了九个协作智能体构成的知识图谱富化框架，实验总计覆盖 1200 篇 PubMed 文章。其中，基因组领域 720 篇文章与 DeepSeek-v3 配置对应 38230 个新实体，以及 83.1%的 LLM 评审通过比例；后一个指标不是人工准确率，也不是全部领域的总体正确率。论文中的 18.6%表示冲突处理阶段移除的新抽取边比例，不能直接解释为相对基线的准确率提升。[16] 这些结果说明该流程在论文设置下具有扩展能力，但本地实现仍须独立复测。多智能体的价值应通过消融和端到端评测判断，不能仅以代理数量衡量。

**表三 三类知识抽取思路的比较**

| 方法 | 核心设计 | 优势 | 需要注意的边界 |
| --- | --- | --- | --- |
| GoLLIE | 训练模型遵循标注指南 | 增强未见模式的零样本泛化 | 包含微调贡献 并非纯提示对照 |
| EDC | 抽取 定义 规范化分阶段完成 | 兼顾开放发现与模式对齐 | 阶段误差会传递 规范化需独立验证 |
| KARMA | 多智能体协作完成图谱富化 | 任务分工清晰 支持冲突处理 | 需审查证据链 自评偏差和代理间锚定 |

### 比较结论

三类方法可以形成互补关系：GoLLIE 提示应把标签边界写成可执行指南，EDC 提示应把开放发现与规范化拆开，KARMA 则说明多阶段协作需要证据链和独立验证。对本地项目最有价值的方向不是照搬某一种框架，而是把指南式定义、阶段式处理和多智能体分工组合起来，并用统一评测集检查误差是否在阶段间累积。

## 第八章 KARMA 项目 Prompt 案例分析

### 现有 Prompt 的总体特征

项目分析依据本地实体抽取、关系抽取代码，以及 2026 年 9 月 9 日的 Prompt 对照记录。论文原始场景为生物医学知识图谱，而本地示例已包含控制器、ECU 和传感器等技术文档。新版 Prompt 使用 system 与 user 区分规则和输入，明确源文本是事实依据，要求保留否定、情态、条件和数量，并在第二阶段生成增删改操作。因此下面的问题主要是仍需评测的风险与改进假设，并非已证实的新版本缺陷；对照记录本身也注明未调用 API，不构成效果实验。[23]

## 规则冲突与判定优先级

旧版中的原子化要求与完整短语要求容易被混淆。新版已分别规定保留必要的实体修饰语，以及把多个独立动作拆成多条事实；这两条原则可以同时成立。例如“the measured traffic per time”可保留为一个完整参数表达，而“reads and stores sensor data”可拆成两条关系。后续应检查模型是否仍把“原子事实”误解为“最短词组”，并用边界样例验证规则优先级。精确率和召回率的取舍也应落实为评测门槛，而不是在 Prompt 中同时无条件要求二者最大化。

## 两阶段修订的锚定风险

第二阶段同时看到源文本、首轮结果和差异信息，为修复遗漏提供了条件，也带来一个待验证风险：审校模型可能过度依赖首轮候选，没有重新搜索全文。可设计“独立重抽取后裁决”的对照组，先隐藏旧结果独立抽取，再与旧结果比较；裁决阶段为保留、删除和新增项核对文本证据。该方案会增加调用成本，是否降低锚定、是否引入更多误抽，需要与现有直接审校流程比较，不能提前认定一定更好。

## 自评指标的局限

项目要求抽取模型同时给出 confidence、clarity 和 relevance，分别描述源文支持强度、表达清晰度与主题相关度。新版已说明分数不替代事实核实，这一定位应在下游筛选中保留。尤其是“may query”可以被原文明确支持，因此支持度高不等于事件已经发生。分数是否适合排序或设阈值，需要与独立人工标注比较；只有先定义它所对应的概率事件，才适合讨论概率校准。模型自述的 0.9 不能直接解释为 90%的客观正确概率。

## 需要单独覆盖的语言现象

技术文档和生物医学文本都需要单独评估否定、推测、条件、数量、并列、比较和指代。比如“may query”不能直接变成“queries”，“A 和 B 分别作用于 C 和 D”不能展开为四条关系，“未观察到 X 导致 Y”也不能抽为肯定因果。新版已有相关要求，下一步应建立挑战集，观察它们在复杂长句中是否仍能执行。还需检查下游模式对齐、摘要和冲突处理是否保留这些限定，避免前端抽取正确、后端规范化时又丢失语义。

## 一个可用于讲解的完整案例

使用项目对照记录中的示例“The controller reads and stores sensor data. It may query the ECU.”，可以同时说明三种设计要求。第一句支持两个独立事实，即 controller 读取 sensor data 与存储 sensor data；

第二句的关系必须保留 may query，而不是改成无条件的 queries。按照当前保留原文表达的策略，头实体可以保留 It，并附上原文证据；如果后续需要将 It 消解为 controller，应单独记录指代解析结果，而不把改写内容冒充原文跨度。此处是规则解释示例，不是本报告实际运行模型得到的实验结果。[23]

评测时可分别检查独立动作是否漏抽、may 是否丢失、It 的处理是否符合约定、证据片段是否来自原文，以及结构是否可解析。这样，一个看似简单的案例就对应数个可测约束。若只能展示一页项目案例，建议使用“源文本—不符合规则的候选—符合约定的结果—对应评测项”的顺序，让老师直接看到 Prompt 规则如何转化为检验标准。

## 建议的 Prompt 重构框架

面向 KARMA 的 Prompt 可重构为六个连续模块：任务定义说明抽取对象；术语定义限定实体和关系边界；决策流程规定先证据后归类；冲突优先级解决原子性与完整性矛盾；输出模式保证接口稳定；自检清单检查否定、模态、跨度和无证据补全。每个模块都应能对应至少一类评测用例。若某条规则无法对应可观察错误或测试样例，它很可能只是增加长度而没有增加控制力。

## 建议的对照实验

实验应从可回答的假设出发，而不是一次性替换全部 Prompt。优先选择约 100 至 200 条人工复核样例，保持模型、温度和解析流程一致，分别比较基线 Prompt 与单因素改动。每组实验重复多次并报告平均值与波动，同时记录逐样例变化。这样既能找到有效规则，也能识别看似提高总分、实则伤害关键边界类型的改动。

**表四 KARMA 优先实验矩阵**

| 实验问题 | 对照改动 | 主要指标 | 预期判断 |
| --- | --- | --- | --- |
| 规则优先级是否减少边界冲突 | 原 Prompt 对比分层决策 Prompt | 实体跨度 F1 过度拆分率 | 判断原子性与完整性是否更一致 |
| 独立重抽取是否降低锚定 | 直接修订 对比先独立抽取再裁决 | 新增正确项 删除错误项 召回率 | 判断二阶段流程是否真正发现首轮遗漏 |
| 模态反例是否抑制事实化 | 无反例 对比加入少量正反例 | 模态误判率 否定误判率 | 判断示例是否优于继续增加抽象规则 |
| 结构约束是否只改善格式 | 提示返回 JSON 对比模式约束 | 模式通过率 语义 F1 | 区分接口收益与任务收益 |
| 自评分能否预测正确性 | 模型自评分与人工标签对齐 | 相关系数 校准误差 | 决定自评分适合筛选还是仅作辅助 |

# 第九章 核心结论与认知升级

## 从技巧集合到系统工程

本阶段最重要的认知升级是将模型行为发生的完整条件纳入设计。指令负责目标，数据提供证据，示例校准边界，模式规定接口，工具执行确定性操作，评测验证效果。具体组件应按任务复杂度与风险配置，简单分类不一定需要检索或多智能体。判断工程是否成熟的标准，是所采用的组件是否各自解决明确问题，以及修改后是否有可重复的证据支持。

## 从追求完美 Prompt 到建立证据循环

不存在脱离模型、任务和数据分布的通用最佳 Prompt。更合理的目标是建立一条快速而严谨的证据循环：收集失败样例，形成错误分类，提出单一改动假设，运行固定评测，检查收益与回退，再把新失败加入回归集。Prompt 在这一循环中只是可调组件之一。自动 Prompt 优化研究进一步说明，候选生成、搜索和评估可以部分自动化，但自动化仍依赖可靠目标函数，否则系统只会更高效地迎合错误指标。[10]

## 从输出流畅到结果可信

语言模型最容易制造的错觉是把表达质量误认为事实质量。结构完整、解释顺畅或置信度很高，都不能代替证据核对。对 KARMA 而言，每个实体和关系都应尽可能回指原文跨度或句段，规范化结果要与原始证据分开保存，推测性结论要保留模态，模型自评分要与外部标注校准。只有把可追溯性纳入输出设计，知识图谱才不会把语言生成的不确定性固化为数据库事实。

## 常见误区

常见误区包括把角色设定当作主要质量来源、认为 Prompt 越长越专业、把思维链当作所有任务的默认配置、把 JSON 合法等同于内容正确、用少数成功案例替代评测，以及把 LLM 评审当作没有偏差的金标准。更好的判断方式是询问：这段设计解决了哪一种可观察错误，是否有数据证明，是否带来成本或延迟代价，模型或数据变化后是否仍然成立。

## 当前学习重点排序

若时间只有一周，学习应优先完成任务规格与评测体系，其次研究少样本、结构化输出和上下文工程，再学习推理、工具与自动优化。对本项目最值得精读的是与信息抽取、标注指南、开放模式、

长上下文和评审偏差直接相关的论文；阅读时以问题为单位抓住方法、实验和局限，不必从头到尾平均用力。

# 第十章 后续学习与组会汇报设计

## 一周学习与实验安排

后续一周建议采用“概念建立—方法精读—项目诊断—小规模实验—汇报整合”的节奏。前两天完成官方文档和核心论文的结构化阅读，第三天建立 KARMA 错误分类与人工复核样例，第四至第五天运行一到两个单因素对照实验，第六天整理图表和案例，第七天压缩为组会叙事。每天都应产生可复用产物，例如一页论文卡片、一组失败样例或一张实验表，而不是只记录阅读时长。

## 组会 PPT 的中心叙事

PPT 不宜按网站或论文逐个介绍，而应围绕一个中心判断展开：Prompt 工程已经从提示词技巧发展为面向模型行为的系统工程，评测是连接设计与可靠性的核心，而 KARMA 提供了一个可以验证这些原则的真实场景。建议先用一个失败案例制造问题，再给出概念框架与代表方法，随后展示 KARMA 诊断和实验设计，最后总结可执行的下一步。这样老师看到的不是资料汇编，而是从学习到研究问题的转化。

**表五 建议的组会 PPT 结构**

| 页次 | 页面主题 | 要回答的问题 | 建议素材 |
| --- | --- | --- | --- |
| 1 | 问题与目标 | 为什么普通 Prompt 教程不够 | 一个 KARMA 失败案例 |
| 2 | 核心定义 | Prompt 工程到底是什么 | 任务规格到工程闭环示意 |
| 3 | 方法地图 | 常用方法分别解决什么问题 | 方法家族比较表 |
| 4 | 上下文与工具 | 为什么不只是改措辞 | 检索 工具 结构输出流程 |
| 5 | 评测框架 | 如何证明 Prompt 更好 | 数据集 指标 错误分类 |
| 6 | 安全与鲁棒性 | 系统为何会在真实环境失效 | 提示注入和分布变化案例 |
| 7 | 知识抽取研究 | 代表工作带来什么启示 | GoLLIE EDC KARMA 对比 |
| 8 | 本地 Prompt 诊断 | KARMA 当前最值得改什么 | 规则冲突 锚定 自评偏差 |
| 9 | 实验方案 | 下一步如何验证 | 单因素对照实验矩阵 |

| 页次 | 页面主题 | 要回答的问题 | 建议素材 |
| --- | --- | --- | --- |
| 10 | 结论 | 本阶段真正学到了什么 | 三条核心结论与行动计划 |

## 预期研究产出

如果按上述方案推进，本阶段可形成四类产出：一套可复用的 Prompt 任务规格模板，一份面向知识抽取的错误分类表，一个经过人工核验的小型回归评测集，以及至少一组可复现的 Prompt 对照实验。它们比单次汇报更有长期价值，因为后续更换模型、增加实体类型或调整多智能体流程时，仍可用同一套标准验证变化。

## 汇报时需要主动说明的边界

组会中应主动区分文献结论、本地观察与尚未验证的假设。论文报告的数据只能说明其特定模型、数据和评测设置下的结果；对 KARMA 现有 Prompt 的判断来自代码与规则分析，仍需人工标注和对照实验确认；建议方案则属于下一阶段研究假设。提前说明这三层证据，可避免把综述写成定论，也便于老师针对实验设计而非措辞细节提出建议。

## 展示材料的取舍原则

PPT 应保留能支撑中心论点的图、表和案例，详细术语与文献列表放在报告或备份页。方法介绍每类只保留一个代表机制，KARMA 部分则用原文片段、旧输出和期望输出构成对照案例。若汇报时间有限，应优先讲清评测闭环与两个最重要的项目问题，而不是压缩字号以容纳所有 Prompt 技巧。

# 附录 术语速查与学习资源

## 核心术语

下列术语用于统一后续学习和汇报表达。Prompt 指提供给模型的全部输入组织，而不只是一句用户问题；上下文工程指选择、组织和管理模型决策时可见的信息；结构化输出指用模式约束响应外形；评测集是用于稳定比较版本的数据样本；回归测试用于确认修复新问题时没有重新引入旧问题；提示注入是不可信数据试图改变系统指令的攻击方式。

| 术语 | 简要解释 | 在 KARMA 中的对应对象 |
| --- | --- | --- |
| 任务规格 | 目标 约束 成功标准与异常处理的统一描述 | 实体和关系的定义及判定优先级 |
| 上下文工程 | 管理模型一次决策可见的信息 | 原文 初始结果 差异信息与指南的组织 |
| 少样本提示 | 用少量示例校准标签边界和输出行为 | 复合实体 否定 模态和并列关系示例 |
| 结构化输出 | 用模式保证字段和类型稳定 | 实体列表 三元组及其证据字段 |
| LLM 评审 | 用模型对候选输出进行评分或比较 | 正确性 清晰度和相关性评估 |
| 回归集 | 长期保留的已知失败样例集合 | 每次 Prompt 改动必须重新通过的样例 |

## 推荐学习资源

学习资源应按问题使用，而不是从头浏览。需要核对当前接口和推荐实践时优先看模型厂商官方文档；需要建立方法地图时看 Prompt Report 与 Prompting Guide；需要深入推理方法时阅读 CoT、Self Consistency、Least to Most、PAL 与 ReAct 原论文；需要处理知识抽取时重点阅读 GoLLIE、EDC 和 KARMA；需要设计实验时参考官方 Evals 指南与 Promptfoo；需要审查安全时查看 OWASP 提示注入资料。每次阅读建议记录“解决的问题、核心机制、实验条件、局限、与 KARMA 的关系”五项，而不是摘抄全文。

| 资源 | 适合解决的问题 | 链接 |
| --- | --- | --- |
| OpenAI Prompt Engineering | 当前提示设计与模型使用建议 | https://developers.openai.com/api/docs/guides/prompt-engineering |
| OpenAI Evals | 建立数据集 指标与持续评测 | https://developers.openai.com/api/docs/guides/evals |

| 资源 | 适合解决的问题 | 链接 |
| --- | --- | --- |
| Prompting Guide | 快速查阅方法分类和示例 | https://www.promptingguide.ai/ |
| Learn Prompting | 按课程顺序补充基础与安全 | https://learnprompting.org/docs/introduction |
| Promptfoo | 把 Prompt 对照实验转成自动化测试 | https://www.promptfoo.dev/docs/guides/ |
| OWASP LLM Prompt Injection | 建立提示注入威胁模型 | https://genai.owasp.org/llmrisk/llm01-prompt-injection/ |

# 参考文献

参考文献覆盖方法综述、推理技术、知识抽取、长上下文、评审偏差、安全规范与官方实践。后续精读时应以研究问题为主线核对原始实验，不把论文标题或摘要中的结论直接推广到当前模型与项目数据。


[1] Schulhoff S 等. The Prompt Report: A Systematic Survey of Prompting Techniques. arXiv:2406.06608, 2024. https://arxiv.org/abs/2406.06608

[2] Wei J 等. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS, 2022. https://arxiv.org/abs/2201.11903

[3] Wang X 等. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR, 2023. https://arxiv.org/abs/2203.11171

[4] Zhou D 等. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. ICLR, 2023. https://arxiv.org/abs/2205.10625

[5] Gao L 等. PAL: Program-aided Language Models. ICML, 2023. https://arxiv.org/abs/2211.10435

[6] Lu Y 等. Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity. ACL, 2022. https://aclanthology.org/2022.acl-long.556/

[7] Yao S 等. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023. https://arxiv.org/abs/2210.03629

[8] Sainz O 等. GoLLIE: Annotation Guidelines Improve Zero-Shot Information-Extraction. ICLR, 2024. https://arxiv.org/abs/2310.03668

[9] Zhang B, Soh H. Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction. EMNLP, 2024. https://aclanthology.org/2024.emnlp-main.548/

[10] Zhou Y 等. Large Language Models Are Human-Level Prompt Engineers. ICLR, 2023. https://arxiv.org/abs/2211.01910

[11] Liu N F 等. Lost in the Middle: How Language Models Use Long Contexts. TACL, 2024. https://arxiv.org/abs/2307.03172

[12] Mizrahi M 等. State of What Art? A Call for Multi-Prompt LLM Evaluation. TACL, 2024. https://aclanthology.org/2024.tacl-1.52/

[13] Shi L 等. Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge. IJCNLP-AACL, 2025. https://aclanthology.org/2025.ijcnlp-long.18/

[14] Koo R 等. Benchmarking Cognitive Biases in Large Language Models as Evaluators. Findings of ACL, 2024. https://aclanthology.org/2024.findings-acl.29/

[15] Niklaus C 等. A Survey on Open Information Extraction. COLING, 2018. https://aclanthology.org/C18-1326/

[16] Lu Y, Wu W, Zhao X, Peng R, Wang J. KARMA: Leveraging Multi-Agent LLMs for Automated Knowledge Graph Enrichment. NeurIPS, 2025. 本报告指标释义核对 arXiv v2 全文。 https://arxiv.org/html/2502.06472v2

[17] OWASP. LLM01 Prompt Injection. OWASP GenAI Security Project. https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[18] OpenAI. Reasoning Best Practices. https://developers.openai.com/api/docs/guides/reasoning-best-practices

[19] OpenAI. Evaluation Best Practices. https://developers.openai.com/api/docs/guides/evaluation-best-practices

[20] OpenAI. Structured Outputs. https://developers.openai.com/api/docs/guides/structured-outputs

[21] Anthropic. Prompt Templates and Variables. https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/prompt-templates-and-variables

[22] Google Cloud. Prompt Design Strategies. https://cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/prompt-design-strategies

[23] KARMA 本地项目材料. GPT 执行/20260909_entity_relationship/prompt_comparison.md；karma/agents/entity_extraction/agent.py；karma/agents/relationship_extraction/agent.py. 本地查阅日期 2026 年 9 月 21 日。